Relatório de Refatoração para Testabilidade

Fase 03 – Isolamento com Mockito e Integração Contínua
Sistema de Batalhas PokeSal

1. Objetivo

A Fase 03 adicionou ao sistema regras que dependem de coisas que o código não controla: o sorteio de precisão e de acerto crítico, o horário e o clima que ativam ou desativam o terreno, o salvamento do resultado em um repositório, o registro dos eventos da batalha e o torneio, cujo chaveamento depende do resultado de cada batalha. Os dois requisitos autorais da fase trouxeram mais duas dependências do mesmo tipo: o relógio, para o tempo limite de escolha da ação, e um canal de notificação, para o alerta de HP crítico. Do jeito que o código estava na Fase 02, essas dependências ficariam presas dentro das classes de regra de negócio, e os testes passariam a dar resultados diferentes a cada execução. Este relatório descreve quais classes foram refatoradas para receber esses colaboradores pelo construtor, e por quê.

2. Como estava o acoplamento na Fase 02

Na Fase 02 já existiam dois pontos de aleatoriedade presos no código e uma classe de serviço estática. Nenhum deles podia ser substituído por um objeto de teste:

- **MoveAction** tinha um campo `private static final Random RANDOM = new Random()` para sortear a condição de status do golpe. Por isso, os testes de status só conseguiam ser previsíveis usando chances extremas (0.0 e 1.0); não havia como testar um valor intermediário.
- **BattlegroundCatalog** também criava o seu próprio `Random` estático para sortear o terreno da batalha.
- **DamageCalculatorService** era uma classe utilitária com método estático. Quem chamava (o `MoveAction`) ficava amarrado a essa implementação, e o cálculo não tinha como receber nenhum colaborador.
- **BattleCli** criava as ações com `new MoveAction(move)` e `new ItemAction(...)`. Se as ações passassem a precisar de colaboradores, a CLI teria que conhecer todos eles.
- O efeito do terreno era sempre aplicado, sem considerar horário ou clima, e o resultado da batalha não era salvo em lugar nenhum.

Isso é acoplamento a implementações concretas: a classe de regra de negócio cria ou chama diretamente a classe de baixo nível de que depende. A consequência prática é que não dá para isolar a classe testada, porque junto com ela sempre vem o `Random` verdadeiro, o relógio verdadeiro e assim por diante.

3. Interfaces criadas

Para cada colaborador externo foi criada uma interface, com uma implementação usada no jogo real. Os nomes seguem o padrão em inglês do resto do código; a tabela mostra a correspondência com os nomes usados no enunciado.

Enunciado	Interface	Implementação	Responsabilidade
GeradorAleatorio	`RandomGenerator`	`DefaultRandomGenerator`	Sorteios de precisão, crítico, status, terreno e clima. Recebe um `java.util.Random` pelo construtor.
ServicoTerreno	`BattlegroundService`	`DefaultBattlegroundService`	Diz se o efeito de um terreno está ativo agora. Recebe um `java.time.Clock` e um `WeatherProvider` pelo construtor.
(clima)	`WeatherProvider`	`RandomWeatherProvider`	Informa o clima atual. Como não usamos um serviço de clima real, a implementação sorteia o clima uma vez ao iniciar.
(relógio)	`java.time.Clock`	`Clock.systemDefaultZone()`	Informa o instante atual. O enunciado aceita o próprio `Clock` do Java, que já é uma abstração. É usado pelo serviço de terreno (horário) e pelo tempo limite de escolha da ação (RA-04).
RepositorioResultados	`ResultRepository`	`InMemoryResultRepository`	Guarda o `BattleResult` de cada batalha (vencedor, perdedor, número de turnos e PokeSal usados).
RegistroBatalha	`BattleLogger`	`ConsoleBattleLogger`	Recebe os eventos da batalha (início, ataque, dano, status, terreno, item e tempo esgotado) e os erros que não podem parar a batalha.
(notificação, RA-05)	`TrainerNotifier`	`ConsoleTrainerNotifier`	Envia ao treinador o alerta de HP crítico do seu PokeSal.
(torneio)	`MatchRunner`	`DefaultMatchRunner`	Disputa uma batalha entre dois treinadores e devolve o `BattleResult`. É o que o torneio usa para cada confronto do chaveamento.

Também foram criados quatro tipos de apoio: `DamageResult` (resultado do cálculo de dano: valor, se acertou, se foi crítico e se o terreno alterou o dano), `BattleResult` (resultado de uma batalha encerrada), `BattlegroundConditions` (horário e clima, com o método `isDaytime()`) e o enum `Weather` (SUNNY, CLOUDY, RAINY).

4. Classes refatoradas

Classe	O que mudou	Por quê
`DamageCalculatorService`	Deixou de ser estática. Recebe `RandomGenerator` e `BattlegroundService` no construtor. Agora sorteia a precisão (`nextInt(100) < accuracy`) e o crítico (`nextDouble() < 0.0625`, multiplicando o dano por 1.5), e só aplica o terreno se o serviço disser que ele está ativo. Retorna um `DamageResult` em vez de um `double`.	Precisão e crítico são aleatórios e o terreno depende de horário e clima. Com os colaboradores injetados, um teste pode escolher o valor sorteado e o estado do terreno.
`MoveAction`	Removido o `Random` estático. Recebe `DamageCalculatorService`, `RandomGenerator` e `BattleLogger` no construtor. Envia os eventos na ordem ataque → dano (ou erro) → status → terreno. Se o golpe erra, o dano é zero e o HP do alvo não muda.	Era o principal ponto de aleatoriedade presa no código. Agora a chance de status pode ser testada com qualquer valor, e a ordem dos eventos pode ser verificada.
`ItemAction`	Recebe `BattleLogger` e registra o uso do item.	O uso de item é um dos eventos que precisam ir para o registro de batalha.
`Move`	Ganhou o componente `accuracy` (0 a 100), com validação. Os construtores antigos continuam existindo e usam precisão 100. No catálogo, os golpes com status ficaram com 90.	Regra nova de precisão. Manter os construtores antigos evitou mexer em todos os lugares que criam golpes.
`Battleground`, `HotAsphalt`, `WaterPuddle`	Novo método `isActive(BattlegroundConditions)`, ativo por padrão. `HotAsphalt` só fica ativo de dia (06:00 às 17:59); `WaterPuddle` fica inativa com sol. `CentralFlowerbed` continua sempre ativo.	Cada terreno decide a própria regra, e o serviço só monta as condições atuais. Assim não foi preciso usar `instanceof` no serviço para saber qual terreno é qual.
`BattlegroundCatalog`	`randomBattleground()` passou a receber um `RandomGenerator`.	Remover o segundo `Random` preso no código.

Classe	O que mudou	Por quê
`RunBattleUseCase`	Recebe seis colaboradores: `BattleCli`, `BattlegroundService`, `ResultRepository`, `BattleLogger`, `TrainerNotifier` e `Clock`. No fim de turno, a cura do terreno só acontece se ele estiver ativo, e o dano do status só é registrado quando tira HP. No fim da batalha, salva o `BattleResult`; se o repositório lançar exceção, o erro é registrado e o vencedor continua definido. Mede com o `Clock` o tempo de escolha de cada ação (RA-04) e envia o alerta de HP crítico pelo `TrainerNotifier` (RA-05).	É a classe que conduz a batalha, então é nela que se encontram as regras de terreno, de resultado e os dois requisitos autorais. Tratar as falhas do repositório e do notificador aqui garante que um problema externo não invalida uma batalha que já terminou.
`RunTournamentUseCase` (nova)	Conduz o torneio eliminatório com 4 ou 8 treinadores. Recebe um `MatchRunner` e a `BattleCli` e faz os vencedores avançarem até o campeão.	O torneio depende só do resultado de cada batalha. Recebendo o `MatchRunner`, ele pode ser testado com resultados mockados, sem rodar nenhuma batalha de verdade.
`DefaultMatchRunner` (nova)	Restaura os PokeSal, sorteia o terreno com o `RandomGenerator` e conduz a batalha com o `RunBattleUseCase`.	Separa do torneio o detalhe de como uma batalha é montada e executada.
`PokeSal`, `Passive`	`PokeSal.restore()` devolve HP, atributos e status ao estado inicial, e `Passive.reset()` reinicia as passivas que guardam estado.	No torneio, cada batalha precisa começar com os PokeSal restaurados.
`BattleCli` + `TurnActionFactory` (nova)	A CLI recebe uma `TurnActionFactory` e pede a ela as ações, em vez de usar `new`. A fábrica guarda os colaboradores das ações.	A CLI continua só cuidando da interação com o jogador, sem precisar conhecer gerador, calculadora e registro.
`BattleRunner`	Passou a ser o único lugar onde as implementações concretas são criadas (`new Random()`, `Clock.systemDefaultZone()`, repositório em memória, logger e notificador de console, executor de batalhas) e repassadas aos construtores. Também oferece a escolha entre batalha simples e torneio.	Concentrar a montagem dos objetos no ponto de entrada é o que permite que as classes de regra de negócio dependam só das interfaces.

5. Princípio da Inversão de Dependência (SOLID)

O Princípio da Inversão de Dependência diz que módulos de alto nível não devem depender de módulos de baixo nível: os dois devem depender de abstrações. No PokeSal, os módulos de alto nível são as regras da batalha e do torneio (`DamageCalculatorService`, `MoveAction`, `RunBattleUseCase`, `RunTournamentUseCase`), e os de baixo nível são os detalhes técnicos (sorteio com `java.util.Random`, leitura do relógio do sistema, armazenamento dos resultados, escrita no console e a montagem de uma batalha completa).

Antes da refatoração a dependência apontava para o detalhe: o `MoveAction` criava um `Random` e chamava um método estático do `DamageCalculatorService`. Depois da refatoração, as regras dependem só das abstrações `RandomGenerator`, `BattlegroundService`, `Clock`, `ResultRepository`, `BattleLogger`, `TrainerNotifier` e `MatchRunner`, e as implementações concretas dependem dessas mesmas interfaces ao implementá-las. Quem decide qual implementação usar é o `BattleRunner`, e nenhuma classe de regra de negócio instancia os seus colaboradores. Um efeito visível disso apareceu no SonarQube: as classes de saída novas recebem o `PrintStream` pelo construtor, e o `System.out` só é referenciado no `BattleRunner`.

O ganho para os testes é direto: como os colaboradores chegam pelo construtor, o teste pode entregar um mock do Mockito no lugar de cada um, inclusive com `@InjectMocks`. Por exemplo, um `RandomGenerator` mockado que retorna 0.05 em `nextDouble()` força um acerto crítico, um `Clock` mockado que retorna 22h simula a noite sem depender da hora em que o teste roda, e um `MatchRunner` mockado define quem vence cada confronto do torneio. Também fica possível usar `verify`, `InOrder` e `ArgumentCaptor` sobre o `BattleLogger`, o `ResultRepository` e o `TrainerNotifier`, já que eles são objetos separados com os quais a classe testada conversa.

6. Impacto nos testes

Os testes da Fase 02 de `DamageCalculatorService`, `MoveAction`, `Trainer` (uso de item) e `RunBattleUseCase` foram ajustados para os novos construtores. Nos testes que já existiam, o gerador é configurado para sempre acertar e nunca dar crítico, e o serviço de terreno para manter o terreno ativo, de modo que as asserções da Fase 02 continuaram valendo sem alteração nos valores esperados.

Ao fim da fase, a suíte tem 290 casos de teste em 25 classes, todos passando, e as classes que usam Mockito rodam com `@ExtendWith(MockitoExtension.class)`. A cobertura das classes de regra de negócio é de 91,1% das linhas e 90,6% dos branches, e o build falha se ela cair abaixo de 80% e 70%. Testes, cobertura e Checkstyle rodam na integração contínua a cada pull request. O detalhamento dos testes está no documento de testes.

7. Decisões e pontos em aberto

- A ordem do registro (ataque, dano, status, terreno, item) é seguida dentro de cada ação. O uso de item é registrado quando a ação de item acontece, já que um turno pode ter um ataque e um item, em qualquer ordem, dependendo do SPD.
- A passiva `SolarPower` continua dando o bônus sempre que o terreno for o `HotAsphalt`, mesmo à noite, porque ela depende do tipo de terreno e não do efeito do terreno. Se o grupo decidir que ela também deve desligar à noite, a mudança é pequena.
- No torneio, o chaveamento segue a ordem de inscrição, e os PokeSal são restaurados antes de cada batalha. Os itens gastos continuam gastos durante todo o torneio.
- O `RunBattleUseCase` recebe seis colaboradores. Ainda está abaixo do limite de sete parâmetros da regra java:S107 do SonarQube, mas, se a classe crescer, uma alternativa é separar o controle do tempo limite e o alerta de HP crítico em classes próprias.
- A cobertura é medida só nas classes de regra de negócio: o pacote `io` (console) e o `BattleRunner` (montagem das dependências) ficam fora da meta do JaCoCo.